Small-Agentic-RAG & Quantized Evaluation Bench

Document de conception détaillé — fondations, architecture, expérimentation et feuille de route

Projet personnel de R&D en intelligence artificielle, modèles de langage et systèmes RAG

Résumé exécutif

Small-Agentic-RAG & Quantized Evaluation Bench est un projet de recherche appliquée visant à comprendre, construire et évaluer une chaîne complète de génération augmentée par récupération (Retrieval-Augmented Generation, ou RAG). Le projet suit une approche progressive : d'abord un petit modèle de langage implémenté en PyTorch sert de laboratoire pour étudier certains mécanismes internes des Transformers ; ensuite, un modèle compact pré-entraîné est utilisé pour construire un système Agentic RAG réaliste, expérimenter la quantification et mesurer les performances d'inférence.

Le but n'est pas d'entraîner depuis zéro un modèle de plusieurs milliards de paramètres. Le but est de comprendre les mécanismes importants, de comparer plusieurs variantes de façon contrôlée, et de présenter des résultats reproductibles concernant la mémoire, la latence, le débit et la qualité.

1. Problématique et motivation

Les grands modèles de langage (LLM) peuvent produire du texte cohérent, mais ils ont plusieurs limites pratiques : leurs connaissances internes peuvent être incomplètes ou obsolètes, ils peuvent inventer des informations, et leur exécution peut demander beaucoup de mémoire et de calcul. Un système RAG réduit une partie de ces limites en fournissant au modèle des passages documentaires pertinents au moment de la question. Une architecture agentique va plus loin en décidant quelle action effectuer : rechercher des documents, exécuter un outil ou répondre directement.

En parallèle, des techniques comme Grouped-Query Attention (GQA), Rotary Position Embedding (RoPE), KV-Cache et la quantification influencent l'architecture ou l'efficacité d'exécution des modèles. Le projet cherche à comprendre ce que fait chaque technique et à vérifier expérimentalement ses avantages et ses coûts.

2. Objectifs

- Implémenter un petit Transformer causal en PyTorch pour comprendre ses composants.
- Étudier l'attention standard, GQA et RoPE dans un cadre expérimental contrôlé.
- Implémenter un KV-Cache simplifié et mesurer son effet sur la génération.
- Utiliser des données ouvertes et documenter les licences de réutilisation.
- Construire un pipeline RAG complet avec ingestion, chunking, embeddings et base vectorielle.
- Combiner recherche sémantique dense et recherche lexicale BM25.
- Ajouter un reranker pour améliorer l'ordre des passages candidats.
- Construire un agent capable de choisir entre réponse directe, RAG et outil Python.
- Tester la quantification et un moteur d'inférence optimisé sur un LLM compact pré-entraîné.

- Mesurer la mémoire, la latence, le débit et la qualité de façon reproductible.
- Publier un dépôt propre, des résultats interprétables et une démonstration.

3. Architecture globale : deux niveaux complémentaires

```
PARTIE A — LABORATOIRE EXPÉRIMENTAL
Corpus ouvert
    ↓
Tokenizer → données tokenisées
    ↓
Petit Transformer causal implémenté en PyTorch
    ├── baseline d'attention
    ├── variante GQA
    └── variante avec RoPE
    ↓
Génération autoregressive
    ├── sans KV-Cache
    └── avec KV-Cache
    ↓
Mesures : qualité, latence, tokens/s, mémoire

PARTIE B — SYSTÈME AGENTIC RAG RÉALISTE
LLM compact pré-entraîné
    ↓
Quantification et moteur d'inférence éventuel
    ↓
Question utilisateur → agent de décision
    ├── réponse directe
    ├── outil de recherche RAG
    └── outil Python contrôlé
             ↓
Documents → ingestion → chunking → embeddings
             ↓
          Qdrant / Chroma
             ↓
     Dense Retrieval + BM25
             ↓
           Fusion
             ↓
      Cross-Encoder Reranker
             ↓
Contexte pertinent + question → LLM → réponse
             ↓
Benchmark de bout en bout
```

4. Pourquoi commencer par un petit modèle ?

Un modèle de 7B ou 8B paramètres est trop coûteux à entraîner depuis zéro pour un projet personnel ordinaire. En revanche, un petit modèle causal permet d'implémenter et d'inspecter les calculs essentiels. Il sera entraîné sur un corpus modeste, non pas pour rivaliser avec les LLM modernes, mais pour servir de banc d'essai.

4.1 Ce que fera le petit modèle

- Transformer du texte en tokens puis en vecteurs.
- Apprendre à prédire le prochain token à partir des tokens précédents.
- Générer de courtes séquences de texte.
- Permettre des comparaisons entre plusieurs choix d'architecture.
- Fournir un environnement contrôlé pour mesurer les performances.

4.2 Ce que fera le modèle pré-entraîné

Un LLM compact pré-entraîné sera utilisé dans la seconde partie pour le RAG, l'agent, la quantification et les tests d'inférence réalistes. Le choix final — par exemple dans les familles Mistral, Llama ou Ministral — dépendra de la mémoire disponible, de la licence, du support logiciel et des résultats des premiers essais.

Il ne faut pas modifier arbitrairement l'architecture d'un modèle pré-entraîné. GQA et RoPE sont déjà présents dans de nombreux modèles modernes. Une modification architecturale après entraînement peut rendre les poids incompatibles. Nous implémenterons ces mécanismes dans le petit modèle, puis choisirons un modèle réel qui les utilise déjà lorsque cela correspond à l'expérience.

5. Données et datasets

5.1 Données d'entraînement du laboratoire

Un corpus ouvert et de petite taille sera utilisé pour entraîner le petit modèle. TinyStories est un candidat pédagogique possible, car il contient des histoires simples adaptées à des expériences sur de petits modèles. Avant utilisation ou redistribution, il faudra vérifier les conditions de licence et les conditions d'accès actuelles du corpus.

La tâche est le causal language modeling, aussi appelé prédiction du prochain token. Pour une séquence de tokens x1, x2, ..., xt, le modèle apprend à prédire x(t+1) en se basant uniquement sur les tokens qui précèdent.

$$x_1, x_2, x_3, \ldots, x_t \rightarrow \text{prédire } x_{t+1}$$

5.2 Documents pour le RAG

Le corpus documentaire pourra contenir des documentations techniques, des documents logiciels ou des articles ouverts dont la réutilisation est autorisée. Le projet devra garder une trace de la source, de la licence et des métadonnées de chaque document. Il ne faut pas redistribuer du contenu protégé sans autorisation.

5.3 Données d'évaluation

Pour évaluer le RAG, on préparera un ensemble de questions associées aux documents pertinents et, si possible, à des réponses de référence. On pourra commencer par un petit jeu de test construit manuellement à partir du corpus ouvert, puis l'élargir. Les questions de test ne devront pas être utilisées pour ajuster les résultats de manière opportuniste.

6. Fondations du Transformer

6.1 Tokenisation et embeddings

Le texte ne peut pas être traité directement par les opérations numériques du réseau. Un tokenizer découpe le texte en tokens et associe chaque token à un identifiant entier. Une couche d'embedding transforme ensuite ces identifiants en vecteurs de dimension fixe. Le choix du tokenizer influence la taille du vocabulaire, la longueur des séquences et la complexité de l'entraînement.

6.2 Self-Attention

La self-attention permet à chaque position d'une séquence d'agréger de l'information provenant d'autres positions. Trois représentations sont calculées : Query (Q), Key (K) et Value (V). L'attention standard s'écrit :

```
Attention(Q, K, V) = softmax(QKᵀ / √d□) V
```

De manière intuitive, Q représente ce que la position cherche, K décrit l'information à laquelle elle peut prêter attention et V contient l'information agrégée. Dans un modèle causal, un masque empêche une position de voir les tokens futurs pendant l'entraînement.

6.3 Attention multi-têtes

Multi-Head Attention utilise plusieurs têtes en parallèle. Chaque tête peut apprendre des relations différentes entre les tokens. Cette flexibilité a toutefois un coût en calcul et en mémoire, notamment lorsqu'il faut conserver les clés et valeurs pendant la génération.

7. GQA — Grouped-Query Attention

Grouped-Query Attention réduit le nombre de têtes Key/Value par rapport au nombre de têtes Query. Par exemple, un modèle peut posséder huit têtes Q et deux têtes K/V. Plusieurs têtes Q partagent alors les groupes K/V.

```
Exemple illustratif :
Nombre de têtes Q  = 8
Nombre de têtes K/V = 2

Plusieurs têtes Q partagent les groupes K/V.
Conséquence attendue : moins de K/V à conserver dans le cache.
```

GQA peut réduire la mémoire du KV-Cache et améliorer l'efficacité de l'inférence. Le gain réel dépend toutefois de l'implémentation, du matériel, de la longueur du contexte et de la charge. Les résultats doivent être mesurés plutôt que supposés.

8. RoPE — Rotary Position Embedding

L'attention seule ne suffit pas à représenter naturellement l'ordre des tokens. RoPE encode l'information positionnelle en appliquant des rotations aux représentations Q et K. Cela permet à l'attention de prendre en compte les positions relatives entre tokens.

Dans le laboratoire, une expérience comparera une configuration sans RoPE et une configuration avec RoPE. Il faudra contrôler autant que possible les autres paramètres. On observera notamment la qualité

de prédiction et de génération. RoPE est un choix architectural/positionnel, pas une optimisation équivalente au KV-Cache.

9. KV-Cache : accélérer la génération

Les LLM génèrent généralement le texte de manière autoregressive : ils prédisent un token, puis utilisent ce token pour prédire le suivant. Sans cache, les clés et valeurs des tokens précédents peuvent être recalculées à chaque étape. Le KV-Cache conserve les valeurs déjà calculées.

```
Sans cache :
Étape 1 : calculer K/V pour le token 1
Étape 2 : recalculer K/V pour les tokens 1 et 2
Étape 3 : recalculer K/V pour les tokens 1, 2 et 3

Avec cache :
Étape 1 : calculer et conserver K/V du token 1
Étape 2 : réutiliser le cache et ajouter K/V du token 2
Étape 3 : réutiliser le cache et ajouter K/V du token 3
```

Le cache réduit les recalculs pendant la génération, mais occupe de la mémoire GPU. Le compromis est donc entre le temps de calcul et l'empreinte mémoire. Point essentiel : le KV-Cache est une optimisation de l'inférence autoregressive ; il ne constitue pas une technique d'entraînement standard.

10. Protocole expérimental de la partie 1

La première partie doit être menée comme une expérience scientifique. On commence par une baseline, puis on modifie une variable à la fois autant que possible.

- Baseline : attention standard avec une configuration simple.
- Variante avec RoPE : étudier l'effet de l'encodage positionnel.
- Variante GQA : comparer mémoire et vitesse à configuration comparable.
- Génération sans KV-Cache puis avec KV-Cache.
- Utiliser les mêmes entrées, la même longueur de génération et les mêmes paramètres autant que possible.
- Répéter les mesures, documenter l'environnement et conserver les résultats.
- Mesurer la qualité, par exemple avec la loss de validation ou la perplexité, en plus des performances.

La comparaison entre deux architectures n'est pas toujours parfaitement isolée : changer GQA peut modifier le nombre de paramètres ou les opérations exécutées. Ces différences doivent être décrites dans le rapport.

11. RAG — Retrieval-Augmented Generation

RAG signifie Retrieval-Augmented Generation, ou génération augmentée par récupération. Au lieu de demander au LLM de répondre uniquement à partir de ses paramètres, le système récupère des passages documentaires pertinents puis les ajoute au contexte de génération.

```
Question utilisateur
        ↓
Recherche de passages pertinents
        ↓
```

```
Contexte documentaire + question
        ↓
LLM
        ↓
Réponse fondée sur le contexte récupéré
```

Le RAG permet d'utiliser des informations propres à un corpus et de faciliter la traçabilité des réponses. Il ne garantit toutefois pas automatiquement la vérité : le retrieval peut être mauvais et le modèle peut mal interpréter le contexte.

12. Ingestion, nettoyage et chunking

L'ingestion charge les documents et extrait leur texte. Le nettoyage supprime les éléments inutiles et conserve, si possible, des métadonnées comme le titre, la source, la page ou l'URL. Le chunking découpe ensuite le document en passages appelés chunks.

Des chunks trop petits peuvent perdre le contexte ; des chunks trop grands rendent la recherche moins précise et consomment davantage de tokens dans le prompt. Un chevauchement (overlap) entre chunks peut aider à conserver le contexte autour des frontières. Le projet comparera plusieurs stratégies et mesurera leur effet sur le retrieval et la qualité finale.

13. Embeddings et base vectorielle

Un modèle d'embedding transforme chaque chunk en un vecteur numérique représentant son contenu sémantique. Une question est convertie en vecteur de la même manière. Une base vectorielle, telle que Qdrant ou Chroma, stocke les vecteurs et permet de rechercher les passages les plus proches selon une mesure de similarité.

Les métadonnées doivent être conservées pour permettre le filtrage, l'identification de la source et la présentation de références dans la réponse.

14. Dense Retrieval

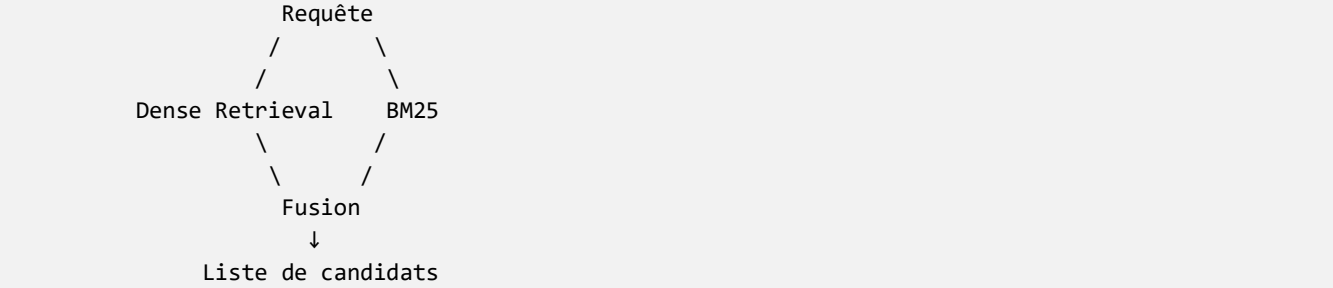
Le Dense Retrieval utilise les embeddings pour retrouver les passages proches sémantiquement. Il est utile lorsque la question et le document expriment une même idée avec des formulations différentes. Sa qualité dépend du modèle d'embedding, de la langue, du domaine et de la préparation des documents.

15. BM25 : recherche lexicale

BM25 est une méthode de recherche lexicale qui attribue un score en fonction des termes de la requête présents dans les documents, de leur fréquence et de la longueur des documents. Elle est souvent efficace pour les noms propres, les identifiants, les codes, les noms de technologies et les mots rares.

Dense Retrieval cherche surtout la proximité sémantique ; BM25 valorise les correspondances lexicales. Les deux méthodes sont complémentaires.

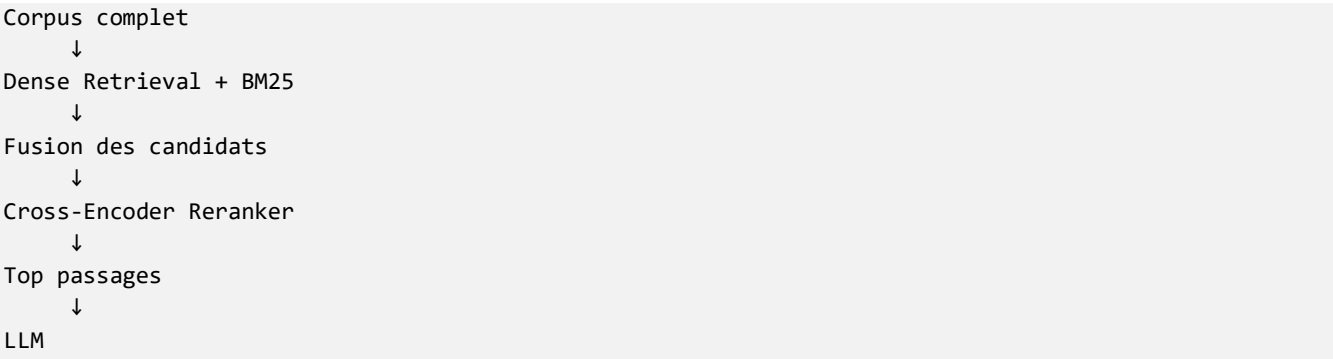
16. Hybrid Retrieval : combiner Dense et BM25



Le retrieval hybride récupère des candidats avec les deux méthodes, puis fusionne leurs résultats. La fusion peut utiliser, par exemple, une méthode de classement réciproque (Reciprocal Rank Fusion) ou une stratégie de score normalisé. Le choix devra être documenté et comparé si le temps le permet.

17. Cross-Encoder Reranker

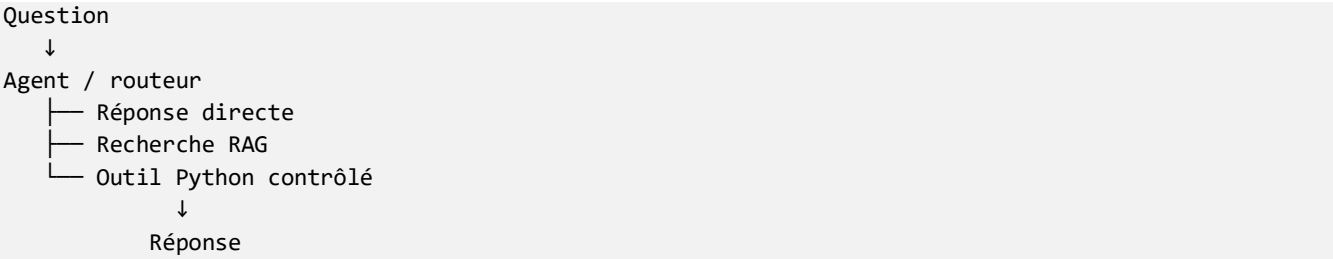
Le retrieval initial doit être rapide et sélectionne un ensemble de candidats. Le reranker intervient ensuite : un cross-encoder prend directement la paire (question, chunk) et produit un score de pertinence plus fin. Les candidats sont alors réordonnés avant d'être transmis au LLM.



Le reranker peut améliorer la précision du contexte, mais il ajoute une latence et un coût de calcul. Le benchmark devra mesurer si le gain de qualité justifie ce coût.

18. Agentic RAG et appel d'outils

Un RAG classique effectue généralement une recherche lorsqu'une question arrive. Dans un Agentic RAG, un agent choisit dynamiquement l'action la plus appropriée à la demande.



- Une question sur les documents peut déclencher la recherche RAG.
- Un calcul numérique peut être confié à un outil Python.
- Une question conversationnelle simple peut recevoir une réponse directe.

- Le function calling représente l'appel d'un outil sous forme structurée, avec des arguments explicites.

Le routeur peut d'abord reposer sur des règles simples ou un schéma de décision limité, puis être remplacé ou amélioré par un modèle capable de sélectionner les outils. Les outils doivent être contrôlés : il ne faut pas exécuter arbitrairement du code fourni par l'utilisateur.

19. Sécurité et fiabilité

Les passages récupérés doivent être traités comme des données non fiables, pas comme des instructions système. Le prompt doit distinguer les consignes de l'application, la question et les extraits documentaires. Le système doit limiter les outils disponibles, valider leurs arguments et éviter l'exécution arbitraire de code. Il faut aussi suivre les sources utilisées et permettre de constater lorsque le contexte ne suffit pas pour répondre.

20. Quantification et AWQ

La quantification représente les poids du modèle avec une précision numérique plus faible. Par exemple, un modèle dont les poids sont en FP16 peut être quantifié en INT8 ou INT4. Cela peut réduire fortement la mémoire nécessaire, mais peut aussi affecter la qualité et les performances.

- FP16 : précision relativement élevée et empreinte mémoire importante.
- INT8 : réduction de mémoire avec une perte de précision généralement modérée selon la méthode.
- INT4 : empreinte plus faible, avec un compromis qualité/performance qui dépend du modèle.
- AWQ signifie Activation-aware Weight Quantization ; c'est une méthode de quantification des poids.
- Le support dépend du modèle, du GPU, du moteur d'inférence et des bibliothèques installées.

La quantification sera principalement étudiée dans la partie consacrée au LLM pré-entraîné. Il faudra comparer les versions avec un protocole identique et noter la méthode exacte, les paramètres et les versions logicielles.

21. LoRA et QLoRA : ne pas confondre les objectifs

LoRA est une technique de fine-tuning efficace en paramètres : de petites matrices entraînables sont ajoutées au modèle afin de l'adapter sans mettre à jour tous ses poids. QLoRA associe ce principe à un modèle de base quantifié pour réduire les ressources nécessaires au fine-tuning. Ces techniques servent principalement à adapter un modèle ; elles ne sont pas synonymes d'AWQ.

22. vLLM, SGLang et Continuous Batching

vLLM et SGLang sont des moteurs d'inférence destinés à exécuter des LLM efficacement. Ils gèrent plusieurs aspects de l'exécution, notamment le traitement des requêtes et la gestion de la mémoire associée au cache. Leur compatibilité dépend du modèle et de l'environnement.

Le continuous batching permet d'ajouter dynamiquement des requêtes à mesure que des emplacements se libèrent, au lieu d'attendre qu'un batch fixe complet se termine. Cela peut améliorer le débit dans des scénarios avec plusieurs requêtes concurrentes. Le bénéfice doit être mesuré sur la charge ciblée.

23. Métriques et protocole de benchmark

Le benchmark est l'une des contributions principales du projet. Les affirmations de performance doivent être accompagnées de mesures reproductibles.

- VRAM / peak VRAM : mémoire GPU utilisée et maximum observé.
- Latence totale : temps écoulé entre la requête et la réponse.
- TTFT (Time To First Token) : délai avant le premier token généré.
- Tokens/s : débit de génération.
- Throughput : volume de requêtes ou de tokens traité par unité de temps.
- Retrieval latency : temps de recherche documentaire.
- Reranking latency : temps consacré au reranker.
- Qualité du retrieval : présence des passages pertinents dans le top-K.
- Qualité de la réponse : exactitude, pertinence et fidélité au contexte.

Pour comparer équitablement les modèles, il faut contrôler la longueur de la requête, le nombre de tokens générés, la taille du contexte, le batch, le matériel et les conditions de cache. Les résultats doivent inclure plusieurs essais et, si possible, une moyenne et une mesure de dispersion.

24. LLM-as-a-Judge

LLM-as-a-Judge utilise un autre modèle pour évaluer les réponses produites. Le juge peut noter l'exactitude, la pertinence et la fidélité au contexte. Cette méthode est utile pour automatiser l'évaluation, mais elle ne fournit pas une vérité absolue : le juge peut être biaisé, incohérent ou sensible à la formulation.

Pour rendre cette évaluation plus fiable, les critères doivent être définis explicitement, les prompts conservés, un sous-ensemble évalué manuellement et les limites clairement rapportées.

25. Tests, reproductibilité et qualité logicielle

- Utiliser pytest pour tester les modules, les formes des tenseurs et les cas limites.
- Vérifier que le KV-Cache produit des résultats cohérents avec la génération sans cache.
- Tester les composants RAG séparément : chunking, embeddings, retrieval, fusion et reranking.
- Tester le routage de l'agent et les erreurs des outils.
- Utiliser des seeds lorsque cela est pertinent.
- Enregistrer les configurations et les versions de dépendances.
- Séparer les données d'entraînement, de validation et de test.
- Conserver les résultats des expériences dans des fichiers structurés.

26. Structure recommandée du dépôt

```
agentic-rag-benchmark/
├── README.md
├── pyproject.toml
├── configs/
├── data/
│   ├── raw/
│   └── evaluation/
```

```
├── src/
│   ├── model_lab/
│   │   ├── attention.py
│   │   ├── rope.py
│   │   ├── gqa.py
│   │   ├── kv_cache.py
│   │   └── train.py
│   ├── rag/
│   │   ├── ingestion.py
│   │   ├── chunking.py
│   │   ├── embeddings.py
│   │   ├── dense.py
│   │   ├── bm25.py
│   │   ├── fusion.py
│   │   └── reranker.py
│   ├── agents/
│   │   ├── router.py
│   │   └── tools.py
│   └── evaluation/
│       ├── metrics.py
│       └── benchmark.py
├── tests/
├── app/
└── notebooks/
```

Cette structure est une proposition de départ, pas une contrainte. Le dépôt peut être simplifié au début puis organisé à mesure que les modules deviennent stables.

27. Feuille de route sur huit semaines

Semaines 1–2 : fondations et laboratoire

- Préparer l'environnement, pyproject.toml et la structure.
- Choisir un corpus et un tokenizer.
- Implémenter un petit Transformer causal minimal.
- Implémenter et tester l'attention standard.
- Ajouter GQA et RoPE séparément.
- Implémenter le KV-Cache pour l'inférence.
- Mesurer qualité, latence, tokens/s et mémoire.

Semaines 3–4 : RAG et agent

- Construire l'ingestion documentaire et le chunking.
- Indexer les embeddings dans Qdrant ou Chroma.
- Ajouter Dense Retrieval et BM25.
- Combiner les résultats par une méthode de fusion.
- Ajouter un cross-encoder reranker.
- Construire un agent simple avec recherche RAG et outil Python contrôlé.
- Tester les réponses et les cas où le contexte est insuffisant.

Semaines 5–6 : modèle réel et optimisation

- Sélectionner un LLM compact compatible avec les ressources disponibles.

- Mesurer une baseline avant optimisation.
- Tester une méthode de quantification compatible.
- Tester vLLM ou SGLang si le matériel et le système le permettent.
- Automatiser les mesures de latence, mémoire et débit.
- Comparer la qualité avant/après quantification.

Semaines 7–8 : publication et démonstration

- Nettoyer le dépôt et documenter l'installation.
- Rédiger les protocoles et les résultats.
- Présenter les compromis et les limites.
- Créer une interface Gradio ou Streamlit.
- Préparer une démonstration publique si les ressources et licences le permettent.
- Rédiger le README, les instructions de reproduction et les perspectives.

28. Tableau récapitulatif des concepts

Concept	Fonction principale	Partie concernée
GQA	Réduit le nombre de têtes K/V et le coût potentiel du cache	Architecture / inférence
RoPE	Encode l'information positionnelle	Architecture
KV-Cache	Réutilise les K/V calculés précédemment	Inférence
Dense Retrieval	Recherche sémantique par embeddings	RAG
BM25	Recherche lexicale par termes	RAG
Hybrid Retrieval	Combine retrieval dense et lexical	RAG
Reranker	Réordonne finement les candidats	RAG
Quantification	Réduit la précision des poids et l'empreinte mémoire	LLM pré-entraîné
LoRA	Adapte un modèle avec peu de paramètres entraînables	Fine-tuning
QLoRA	Associe LoRA à une base quantifiée	Fine-tuning
vLLM/SGLang	Exécute les LLM avec une inférence optimisée	Déploiement
LLM-as-a-Judge	Évalue automatiquement des réponses	Évaluation

29. Ordre conseillé pour le développement

```
1. Environnement et structure du dépôt
2. Corpus + tokenizer
3. Petit modèle causal minimal
4. Attention baseline
5. Entraînement et génération baseline
6. GQA
7. RoPE
8. KV-Cache
9. Benchmarks du laboratoire
```

```
10. Ingestion documentaire
11. Chunking
12. Embeddings + base vectorielle
13. Dense Retrieval
14. BM25
15. Hybrid Retrieval
16. Reranker
17. Agent + outils
18. LLM compact pré-entraîné
19. Quantification
20. vLLM/SGLang si possible
21. Benchmarks finaux
22. Documentation, démo et analyse
```

30. Livrables finaux et valeur R&D

- Un petit Transformer causal documenté, avec des tests.
- Une comparaison contrôlée des variantes d'attention et de cache.
- Un pipeline RAG reproductible.
- Un agent qui choisit entre plusieurs outils de manière traçable.
- Un benchmark de performance avec les configurations enregistrées.
- Une analyse des compromis qualité / mémoire / latence / débit.
- Un dépôt GitHub avec instructions d'installation, limites et résultats.
- Une démonstration interactive, si le matériel le permet.

La valeur du projet ne résidera pas uniquement dans la création d'un chatbot. Elle reposera sur l'association de l'implémentation bas niveau, du retrieval hybride, du reranking, de l'agentic workflow et d'une évaluation mesurée. Le résultat attendu est un projet R&D qui explique non seulement ce qui fonctionne, mais aussi dans quelles conditions, à quel coût et avec quelles limites.

31. Points à décider avant de coder

- Quel matériel est disponible (CPU, RAM, GPU et VRAM) ?
- Quelle taille viser pour le petit modèle expérimental ?
- Quel tokenizer et quel corpus ouvert utiliser ?
- Quel corpus documentaire servira au RAG ?
- Quel modèle compact pré-entraîné est compatible avec le matériel et la licence ?
- Quelles métriques seront obligatoires pour chaque expérience ?

Ces décisions seront prises avant les expériences afin d'éviter de choisir les paramètres après avoir vu les résultats. Le projet commencera ensuite par la baseline la plus simple, puis ajoutera chaque mécanisme progressivement.